Technology Stack

CrediShield AI — Credit Card Approval Prediction System

Complete Technology Stack

Backend & Machine Learning

Technology	Version	Category	Purpose
Python	3.12	Programming Language	Core language for ML pipeline, backend server, and scripting
Flask	3.0.3	Web Framework	Lightweight REST API server and HTML template rendering
Scikit-learn	1.5.0	ML Library	Model training (6 algorithms), preprocessing (LabelEncoder, StandardScaler), evaluation metrics
XGBoost	2.0.3	ML Library	Extreme Gradient Boosting classifier (one of 6 compared models)
Pandas	2.1.4	Data Processing	DataFrame operations, CSV loading, feature engineering, data cleaning
NumPy	1.26.4	Numerical Computing	Array operations, mathematical computations, feature vector assembly
Joblib	1.5.3	Serialization	Efficient saving/loading of trained models, scalers, and encoders to/from disk
imbalanced-learn	0.8+	ML Library	BalancedRandomForestClassifier for handling class imbalance (training only)
Matplotlib	3.4+	Visualization	EDA charts in Jupyter notebook (training only)
Seaborn	0.11+	Statistical Visualization	Heatmaps, distribution plots, correlation matrices in notebook (training only)
Gunicorn	22.0.0	WSGI Server	Production HTTP server for deploying Flask on Render.com

Frontend

Technology	Version	Category	Purpose
HTML5	—	Markup	Semantic page structure using Jinja2 templates
CSS3	—	Styling	Glassmorphism dark theme, animations, responsive layouts, custom scrollbars
JavaScript (ES6+)	—	Client Logic	Form handling, Fetch API calls, localStorage management, SVG animations, PDF generation
Chart.js	4.x (CDN)	Charting Library	Interactive bar, doughnut, and pie charts for analytics dashboard
Google Fonts	Plus Jakarta Sans	Typography	Modern, premium font family for all UI text

DevOps & Infrastructure

Technology	Version	Category	Purpose
Git	Latest	Version Control	Source code tracking, branching, commit history
GitHub	—	Code Hosting	Remote repository, CI/CD trigger for Render auto-deploy
Render.com	Free Tier	Cloud Platform	Web service hosting with auto-deploy from GitHub
Jupyter Notebook	—	Development Tool	Interactive EDA, prototyping, and visualization during research phase

---

Architecture Pattern

Monolithic MVC (Model-View-Controller)

```
┌──────────────────────────────────────────────────────┐
│                                                      │
│   VIEW              CONTROLLER         MODEL         │
│                                                      │
│  ┌──────────┐     ┌──────────┐     ┌──────────┐     │
│  │index.html│────►│ app.py   │────►│ML Engine │     │
│  │style.css │◄────│(Flask)   │◄────│(sklearn) │     │
│  │script.js │     │          │     │          │     │
│  └──────────┘     └──────────┘     └──────────┘     │
│                                                      │
└──────────────────────────────────────────────────────┘
```

- **View**: `index.html` (structure), `style.css` (theme), `script.js` (interactivity)
- **Controller**: `app.py` (Flask routes handle requests, preprocessing, and responses)
- **Model**: Serialized ML artifacts (`.joblib` files) — pre-trained, inference-only

---

Why These Technologies Were Chosen

Decision	Options Considered	Chosen	Rationale
Language	Python vs. R vs. Java	Python	Best ecosystem for ML (scikit-learn, pandas). Flask is lightweight for REST APIs. Huge community support.
Web Framework	Flask vs. Django vs. FastAPI	Flask	Lightest footprint. No ORM overhead needed. Jinja2 templates built-in. Perfect for single-page apps.
ML Library	Scikit-learn vs. TensorFlow vs. PyTorch	Scikit-learn	Tabular data → traditional ML outperforms deep learning. Built-in preprocessing, metrics, and model selection.
Frontend	React vs. Vue vs. Vanilla JS	Vanilla JS	No build step needed. Simpler deployment. Direct DOM manipulation for SVG animations. Chart.js CDN for charts.
Styling	Tailwind CSS vs. Bootstrap vs. Vanilla CSS	Vanilla CSS	Full control over glassmorphism effects, custom animations, and theme variables. No framework dependencies.
Database	PostgreSQL vs. SQLite vs. localStorage	localStorage	No database setup required. Sufficient for demo-scale prediction history. Easily upgradeable later.
Deployment	Render vs. Vercel vs. Heroku vs. AWS	Render	Free tier with full server (not serverless). Supports Python/Flask natively. Auto-deploy from GitHub. No bundle size limits.

WSGI Server	Gunicorn vs. uWSGI vs. Waitress	Gunicorn	Industry standard for Flask. Simple configuration via Procfile. Compatible with Python 3.12 (v22.0.0).

Dependency Separation Strategy

To optimize deployment size and prevent version conflicts:

File	Purpose	Packages
`requirements.txt`	**Runtime dependencies** (deployed to Render)	flask, numpy, pandas, scikit-learn, xgboost, joblib, gunicorn
`requirements-train.txt`	**Training dependencies** (local development only)	All runtime + matplotlib, seaborn, imbalanced-learn

Rationale: Matplotlib and Seaborn add ~350MB to the bundle (including their C dependencies). Since they're only used in `train.py` and `notebook.ipynb` (not by the Flask server), excluding them from production requirements keeps the Render deployment lean.

Project Name: CrediShield AI
Author: Dinesh (dineshTech-creator)
Date: July 2026